Proof-of-Useful-Work: Integrating Deep Learning into Blockchain Mining

Advanced Study Project II (Vertiefungsarbeit II)

in the Master of Science in Engineering: Data Science program

Submitted by

Vincent Stadler

on January 31, 2025

at ZHAW: Institute of Data Analysis and Process Design

Supervised by

Dr. Christian Badertscher

1. Introduction

Blockchain technologies have made significant advancements in recent years and provide a foundation for secure, decentralized systems. However, the energy consumption of traditional Proof-of-Work (PoW) systems, such as Bitcoin, remains a major point of criticism. These systems require computationally intensive tasks, such as solving hash puzzles, which offer no direct benefit outside of network security. This raises the question of whether the mining process can be designed in a more meaningful way without compromising the security guarantees of these systems.

This project aims to develop an alternative to classical PoW systems by integrating useful work into the mining process. Specifically, it explores how machine learning model training can be incorporated as part of mining to ensure both network security and the generation of productive results. This approach leads to what is known as Proof-of-Useful-Work (PoUW), where computing resources are not only used for cryptographic tasks but also for training machine learning models, particularly deep-learning models. This concept, referred to as Proof-of-Deep-Learning (PoDL), forms the core of this work.

A key component of this project was the development of a method to predict when training a machine learning model can still be considered useful work. This assessment is based on analyzing the trajectory of the loss function and forecasting its future values using a Transformer model trained specifically for this purpose. The goal of this model is to accurately predict the next values of the loss sequence.

A separate algorithm was developed to identify the saturation point of training based on the loss values. This algorithm determines when training no longer provides additional benefits and thus becomes inefficient. By combining loss value forecasting and saturation point detection, inefficient training phases can be identified and avoided early, ensuring that computational resources are allocated only to useful work rather than being wasted on ineffective training processes.

A key reference for this project is the framework proposed by Xiangyu Su and Mario Larangeira [1], which provides a foundation for integrating machine learning tasks into the mining process. The system developed in this project integrates into this framework and expands the possibilities of making the mining process more efficient and sustainable. In doing so, the project contributes to the advancement and optimization of PoUW systems.

2. Background & Motivation

2.1 Blockchain Technologies

Blockchain technologies have introduced a transformative approach to securing, recording, and sharing data in decentralized networks. At their core, blockchains function as distributed ledgers, enabling multiple participants to maintain a synchronized, transparent, and tamper-resistant record of transactions without relying on a central authority. This innovation, first outlined in Satoshi Nakamoto's 2008 Bitcoin whitepaper [2], provided the foundation for cryptocurrencies and other decentralized applications.

Central to the functioning of many blockchains is the PoW consensus mechanism. In PoW systems, participants known as miners compete to solve cryptographic puzzles by performing computationally intensive tasks. The first miner to solve the puzzle validates the next block of transactions, which is then appended to the blockchain. This process secures the network by making it computationally expensive for malicious actors to alter the ledger.

PoW's decentralized security model was revolutionary as it allowed participants to achieve consensus in a trustless environment without relying on intermediaries. However, this mechanism comes with inherent limitations.

2.2 Criticisms of Traditional PoW Systems

While PoW has proven effective in securing blockchain networks, it has also attracted criticsm due to its high energy consumption and computational inefficiency. The process of solving cryptographic puzzles is designed to be computationally demanding, ensuring the difficulty scales with the growth of the network. This feature, while essential for security, leads to a resource-intensive race among miners.

Studies, such as those by de Vries [3] have highlighted that Bitcoin mining alone consumes as much energy as some small countries, with the majority of this energy being sourced from non-renewable resources. This energy consumption raises concerns about the environmental sustainability of PoW systems.

Another significant critique lies in the lack of utility in the computational work performed by miners. The cryptographic puzzles, while effective for securing the network, serve no purpose beyond this function. This has raised questions about the broader societal and economic value of expending such significant computational resources on tasks with no external utility.

These limitations – excessive energy use and the absence of useful outcomes – have motivated researchers to explore various alternatives such as Proof-of-Stake, Proof-of-Authority or Proof-of-Deep-Learning. This project specifically focuses on Proof-of-Deep-Learning.

2.3 Proof-of-Deep-Learning

The Proof-of-Deep-Learning and its extended distributed variant, Distributed Proof-of-Deep-Learning (D-PoDL), are promising frameworks for addressing key limitations of traditional PoW systems. They replace the resource-intensive cryptographic puzzles of PoW with useful computational tasks connected with the training of deep learning models. As mentioned above, traditional PoW systems suffer from excessive energy consumption, and a lack of meaningful output beyond securing the blockchain. The D-PoDL framework, as outlined by Su et al., addresses these issues by allowing miners (or provers) to perform deep learning model training as their proof of work. This shift introduces external value to the consensus mechanism, as the trained models can be applied to real-world domains.

Key Innovations of D-PoDL:

1. **Hash-Training-Hash Structure:** D-PoDL introduces a hash-training-hash structure, ensuring task integrity and preventing miners from engaging in grinding attacks (cherry-picking advantageous initial parameters). This mechanism enforces fairness and

maintains computational difficulty, similar to traditional PoW systems, while adapting it to the context of deep learning.

2. **Model Referencing and Distributed Training:** Unlike prior schemes, D-PoDL incorporates a model-referencing mechanism, which enables miners to train atop pre-trained models (intermediate models). This feature fosters collaboration among miners by allowing them to iteratively improve the same task while earning rewards for their contributions. As a result, even partially trained models are preserved and rewarded, ensuring efficient use of computational resources.
3. **Generic Blockchain Protocol with Robust Security Properties:** D-PoDL integrates a generic blockchain protocol that supports both the traditional longest-chain rule and a weight-based chain selection framework. This flexibility enhances security by allowing protocols to assign weights to blocks based on their contribution quality, ensuring better alignment with the goals of the distributed system.

By integrating deep learning tasks into the consensus mechanism, D-PoDL offers a path to overcoming aspects of the blockchain trilemma [4]. The framework preserves security through computational intensity, maintains decentralization by allowing miners to collaborate on distributed tasks, and enhances scalability by optimizing resource use through model referencing.

By addressing key challenges such as task distribution, collaborative training, and robust security analysis, it builds on prior schemes like DLchain, Proof-of-Learning, and CoinAI [1].

3. Objectives and Research Question

3.1 Objectives of the Project

Traditional PoW systems provide network security at the cost of high computational power without providing external utility beyond the validation of blocks. PoUW aims to address this inefficiency by redirecting computational effort toward tasks with real-world value, such as training machine learning models. However, a fundamental challenge arises in defining when computational work remains useful – particularly in deep learning training, where improvement does not always follow a linear trajectory.

The primary goal of this project is to enhance the D-PoDL framework prototyped by Su et al., particularly addressing the limitations of the current method for determining when training a machine learning model can still be considered "useful work." The existing framework relies on a hard threshold for training accuracy to decide when the training should stop. However, this approach raises several critical questions:

- **Is training accuracy the right metric?** While training accuracy indicates how well the model performs on the training dataset, it does not reflect the model's generalization capabilities, which are crucial in real-world applications.
- **How should the stopping threshold be determined?** A single fixed stopping threshold may be ineffective because training progress is non-linear, with potential plateaus, oscillations, or temporary declines before improvement.

- **How can the threshold be generalized across different tasks?** Deep learning models vary in architecture, loss functions, and training behavior. The stopping criteria must adapt to different setting instead of relying on a hard-coded threshold.

This project aims to integrate these considerations into the D-PoDL framework by developing a prototype to analyze and predict the evolution of a models training metric to dynamically determine the point at which the training process transitions from being productive to inefficient.

3.2 Research Question

To address these challenges, the central research question of this project is:

How can we determine when deep learning training remains useful work in D-PoDL systems?

This question involves defining "useful work" and proposing a framework to determine the optimal stopping point for deep learning trainings. The results of this research aim to refine the D-PoDL framework, making it more adaptable and applicable to real-world use cases.

4. Methodology

4.1 Defining Useful Work

Useful work in the context of D-PoDL refers to the training of deep learning models where the computational effort results in meaningful improvements to the model's performance. Determining when training qualifies as useful work requires a nuanced analysis of metrics, their development over time, and their relationship to the task at hand.

The choice of metrics is inherently task-dependent. For classification tasks, accuracy is the most straightforward metric, but it often lacks nuance, especially in cases of imbalanced datasets where metrics such as precision, recall, and F1-score provide a more comprehensive evaluation. In regression tasks, performance is typically assessed using measures like mean squared error, mean absolute error, or R^2, which evaluate how closely the model predicts continuous values. These task-specific metrics form a basis for evaluating model quality but do not fully capture the overall progression and effectiveness of the training process.

Equally important is the temporal dimension of performance evaluation. Metrics need to be monitored over time to identify trends in the model's learning process. A static threshold, such as a fixed accuracy target, may not be sufficient to capture the dynamics of training. Instead, observing the rate of improvement in accuracy or error reduction provides insights into whether further training is likely to yield meaningful benefits. For example, a plateau or diminishing returns in performance improvement could indicate that the model has reached its practical limits, and additional computation would no longer qualify as useful work.

The figure below illustrates this case in a real training process, where both training and validation loss plateau after about 20 epochs. Despite continued computation, no meaningful reduction in loss is observed beyond this point, showcasing that further training does not contribute to model improvement and thus ceases to qualify as useful work

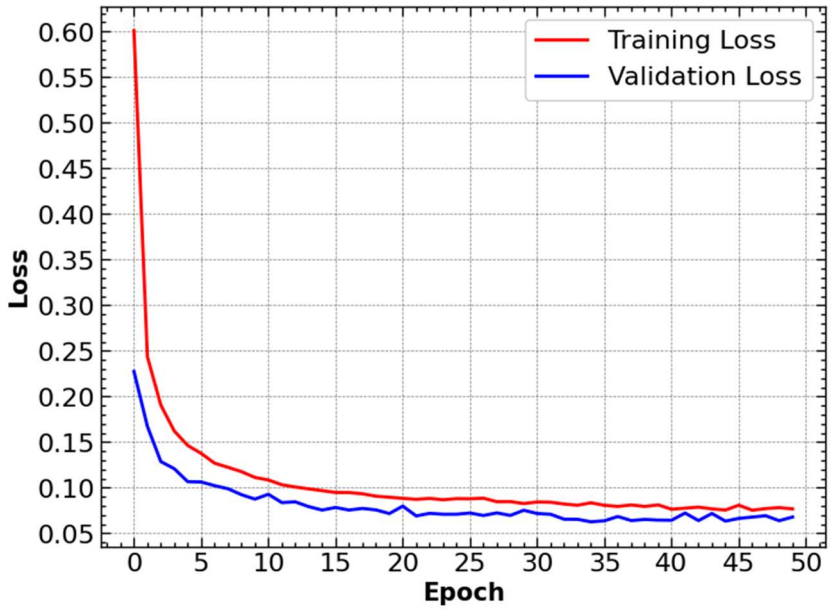

Figure 1: Training and validation loss of a deep learning training [5]

A crucial aspect of defining useful work lies in focusing on test performance rather than solely relying on training metrics. While training metrics indicate how well a model learns from the training dataset, they do not reflect the model's generalization ability, which is essential for real-world applications. Test accuracy, or corresponding metrics for regression tasks, measures performance on unseen data and thus offers a better indication of whether the model solves the problem effectively. This focus on generalization is particularly important in preventing overfitting, where a model performs well on training data but fails to deliver consistent results on new inputs.

Unlike task-specific metrics such as accuracy, F1-score, or R^2, which are tailored to specific types of tasks, the loss function is an integral part of the training process itself. It is universally used to guide optimization and is therefore relevant across both regression and classification tasks. This universality ensures that the method for determining useful work is consistent and not constrained by the specific evaluation criteria of different tasks. By monitoring the evolution of loss values over time, it becomes possible to identify critical stages of training, such as when the model is making meaningful progress versus when improvements plateau [6].

Based on these considerations, I chose to use the evolution of loss values over time as the primary criterion for determining useful work. The loss values, being universally applicable across different tasks and integral to the optimization process, provide a consistent and fine-grained measure of training progress. By analyzing their progression, it is possible to identify when meaningful improvements occur and when the training process reaches a saturation point.

4.2 Implementation Details

4.2.1 Outline

This project builds upon and extends the D-PoDL framework introduced by Su et al., enhancing its flexibility and applicability to diverse training scenarios. The original framework employs a hash-to-hash architecture to ensure computational difficulty and task integrity by linking training tasks to cryptographic hashes. However, their approach to determining when training

qualifies as useful work relies on a hardcoded accuracy threshold tailored to a specific task. This static criterion does not account for the dynamic nature of the training process and cannot be adapted to other tasks or conditions.

In my implementation, the hardcoded threshold is replaced with a more adaptive approach that considers the entire sequence of loss values over time. Furthermore, a predictive model was developed to analyse this sequence and forecast future loss values. By combining historical data with predictive insights, the system can preemptively identify when further training steps are unlikely to yield meaningful improvements. These modifications enhance the current D-PoDL framework by making it applicable to a broader range of deep learning tasks and training conditions while incorporating a more sophisticated understanding of useful work.

4.2.2 Data Collection & Preprocessing

The data used to train the predictor model consists of logs capturing metrics such as training accuracy and training loss values per epoch from previously conducted deep learning training trainingss. These logs are commonly generated by frameworks like TensorFlow and PyTorch, which output training and validation metrics at the end of each epoch. To compile a diverse and comprehensive dataset, these logs were scraped from publicly available repositories on GitHub.

Scraping GitHub presented several challenges due to network restrictions, rate limits imposed by the GitHub API, and access limitations for certain repositories. To overcome these obstacles, a Selenium script was developed. This script mimicked human browsing behavior to bypass network restrictions, navigate through repositories, and extract relevant log data efficiently. The automation process involved identifying and filtering repositories based on specific criteria, such as the presence of TensorFlow or PyTorch-based training logs, ensuring that the collected data was relevant and usable.

The collected training logs are categorized on the following dimensions:

- **Dataset:** MNIST and CIFAR-10 were selected for their contrasting properties – MNIST being simpler with grayscale digit images and CIFAR-10 more complex with color images spanning diverse object classes.
- **Loss Function:** Sparse categorical crossentropy and categorical crossentropy were used to cover distinct optimization behaviors, particularly in handling class probabilities during training.
- **Model Architecture**: Logs were further categorized based on Convolutional Neural Networks (CNNs) and Transformers, which differ in their training dynamics.

This categorization is essential to address the high variance in loss function behavior across different training conditions. Without separating the data along these dimensions, the predictor model would struggle to generalize effectively, as the underlying patterns in the loss values would be obscured by noise introduced by differences in datasets, loss functions, and architectures. By organizing the data in this manner, the predictor model is better equipped to learn accurate and reliable patterns for forecasting loss trends.

Among these dimensions, CNN + CIFAR-10 + Categorical Crossentropy had the largest volume of samples and was prioritized during training and validation for its richness in data.

Details on the data statistics, including a histogram of sequence lengths, the distribution of loss values, and examples of the training data, can be found in the Appendix.

Data Filtering and Sequence Preparation

To ensure meaningful patterns could be learned, sequences were required to contain at least 6 loss values – 5 as input features and 1 as the ground truth (output). Sequences with fewer than 6 values were discarded, as they lacked sufficient historical context for a reliable prediction.

Loss values across all categories were standardized using the global mean and standard deviation. This normalization ensured consistency across datasets, architectures, and loss functions, preventing biases due to varying scales.

The sequences were further augmented by creating overlapping windows. For example, from a sequence of 10 loss values, multiple sub-sequences were generated, starting with at least 5 input values to predict the next loss value. Gradually, additional floats were included in each subsequent sub-sequence, allowing the model to learn patterns across varying input lengths while maintaining the temporal order of the original data. At the end of the preprocessing pipeline our dataset consists of 17’656 training samples.

4.2.3 Model Architecture

To predict the evolution of loss values over time, the implementation employs models designed for time-series data, such as Recurrent Neural Networks (RNNs) and Transformers. While RNNs are well-suited for sequential data due to their ability to model temporal dependencies, Transformers were chosen as the primary architecture in this project due to their superior capacity for understanding long-range dependencies and intricate patterns in time-series data.

Transformers, originally introduced by Vaswani et al. [6], have demonstrated exceptional performance in a wide range of sequence-based tasks. Their use of self-attention mechanisms allows the model to weigh the importance of various time steps, making them particularly effective for capturing complex trends in loss evolution. However, the flexibility and power of Transformers come with a risk of overfitting, especially when working with smaller datasets. To mitigate this, specific architectural and regularization techniques were incorporated.

Transformers were chosen over RNNs primarily due to their scalability and ability to capture long-term dependencies in time-series data. Unlike RNNs, which process sequences sequentially and are prone to vanishing gradient issues [7], Transformers process the entire sequence in parallel, significantly improving training efficiency and performance.

While the risk of overfitting remains a challenge, regularization techniques such as dropout and careful tuning of hyperparameters help mitigate this issue, and help the model to generalize better on unseen data.

Loss Functions

Two custom loss functions were considered for training the predictive model: one designed to ensure valid predictions and another tailored for modeling uncertainty with Gaussian distributions. Each was evaluated for its ability to guide the model toward accurate and meaningful predictions while addressing different aspects of the problem.

The first loss function $\mathcal{L}_1$ was designed to guide the model toward valid predictions where while maintaining simplicity in its optimization. SmoothL1Loss is chosen as the base loss because it balances the robustness of L1 loss with the sensitivity of L2 loss, making it resilient to outliers while effectively penalizing small errors. Additionally, a penalty term discourages invalid negative predictions after de-standardization. Negative values are penalized using a ReLU operation, with the penalty scaled relative to the batch size and base loss to avoid dominating the optimization process:

$$\mathcal{L}_1 = \text{SmoothL1Loss}(\hat{y}, y) + \frac{\text{SmoothL1Loss}(\hat{y}, y)}{10} \cdot \text{ReLU}\big(-(\hat{y} \cdot \sigma + \mu)\big)$$

where:

- $\hat{y}$: Predicted normalized value.
- y: Ground truth value.
- μ, σ: Mean and standard deviation for de-standardization.
- $ReLU(x) = \max(0, x)$: The Rectified Linear Unit function.
- $SmoothL1Loss = \begin{cases} 0.5\,(\hat{y} - y)^2 & if\ |\hat{y} - y| < 1, \\ |\hat{y} - y| < 1 & otherwise \end{cases}$

The second loss function $\mathcal{L}_2$ was specifically designed for tasks involving Gaussian distributions, where the model predicts not just a single float (the next loss value) but also a variance that represents uncertainty:

$$\mathcal{L}_2 = \frac{1}{2}\left[\log(\sigma^2) + \frac{(y - \mu)^2}{\sigma^2}\right]$$

It consists of two components:

- $\log(\sigma^2)$, which penalizes large variances, encouraging the model to make confident predictions
- $\frac{(y-\mu)^2}{\sigma^2}$, which normalizes the squared error by the variance, penalizing large deviations when the variance is small.

While theoretically suitable for capturing uncertainty, this approach proved challenging to optimize. The complexity of jointly optimizing mean and variance led to suboptimal predictions and less stable training.

The final implementation utilized the $\mathcal{L}_1$ function, as it offered greater stability for the given task. Although the $\mathcal{L}_2$ function is well-suited for Gaussian distributions and provides a built-in mechanism for uncertainty estimation, it did not perform well in this application. To still incorporate uncertainty into the predictions, dropout during inference was enabled. By performing multiple stochastic forward passes with dropout, the variability in the model's

predictions can be used as an approximation of its uncertainty. This approach provided a practical and effective solution, which will be discussed in more detail in a later section.

Training

The training process involves predicting the next loss value in a sequence based on a fixed number of preceding loss values. The input consists of sequences of at least 5 floats. This minimum requirement of 5 values is justified by the need for sufficient historical context to allow the model to capture meaningful patterns and trends in the data. Shorter sequences may fail to provide enough information about the underlying dynamics, resulting in less accurate predictions. The model processes these sequences and predicts the subsequent loss value, minimizing the error between the predicted and actual values. The loss is calculated using the $\mathcal{L}_1$ function mentioned above. If the loss wouldn't decrease for more than 3 epochs, the model training would be terminated.

Evaluation

The evaluation strategy incorporates two distinct approaches to test the model's ability to generalize and handle sequential prediction:

- **Ordinary Testing:** The test data (20% of the dataset) is in the same way augmented as the training data by creating overlapping windows, ensuring the model's ability to predict each subsequent value in a controlled setup. The performance is evaluated by comparing the predicted values against the ground truth for each step.
- **Hallucination-Based Testing:** In this approach, the model is provided with the first 5 loss values of a test series. After predicting the next value, the model's own output is used as the input for subsequent predictions. This iterative process simulates a "hallucination" scenario, where the model must rely on its internal predictions to continue generating the sequence. The predicted values are then compared to the ground truth for the entire series to assess the model's ability to extrapolate trends over time.

Confidence Score

To complement the evaluation process, a confidence score is introduced to quantify the model's certainty in its predictions. This is particularly important in scenarios such as hallucination-based testing, where the model relies on its own outputs to predict subsequent values, potentially amplifying uncertainty over time.

The confidence score is calculated based on the variability in the model's predictions, providing a robust measure of uncertainty. During inference, dropout is enabled to generate multiple predictions (e.g., 40) for each input. The predicted value is taken as the mean of the generated predictions. The standard deviation (σ) of these predictions is computed to represent variability. To establish a dynamic threshold for standard deviation, the maximum allowable value (max_std) is determined as the 95th percentile of all standard deviations across the test dataset, accounting for typical variability while excluding outliers.

The normalized standard deviation is then computed as $Normalized\ Std = \frac{\sigma}{max\_std}$. The confidence score is derived using an exponential decay function, $ConfidenceScore =$

$exp(-\alpha \cdot Normalized\ Std)$, where α controls the rate of decay. This approach ensures that small increases in variability result in meaningful reductions in confidence, while the score remains bounded between 0 and 1. Higher scores indicate greater certainty, while lower scores signal increased uncertainty, especially in cases of high variability or model inconsistency.

4.2.4 Determining Saturation Point

To identify the saturation point of a series of loss values, an algorithmic approach was employed. Training a black-box model for this purpose would have required labeled data, which inherently introduces subjective judgment and lacks a unique solution. By contrast, the chosen statistical method provides an objective framework to analyze the dynamics of the series and identify stabilization trends.

The implemented algorithm uses smoothed loss values and computes slopes, curvatures, and other metrics to detect when the series reaches a point where further training is unlikely to yield significant improvement. It evaluates conditions such as stabilization, oscillation, and flat changes to determine this point. Below is an explanation of the key components:

- **Smoothing:** The loss value series is smoothed using a moving average with a specified window size. This reduces noise and highlights the underlying trends in the data.
- **Slope Calculation**: The rate of change between consecutive smoothed values is calculated.
- **Curvature Calculation**: The rate of change of the slope is computed to assess acceleration or deceleration in the series.
- **Stabilization Conditions**: Stabilization is detected if
    - Slopes are consistently below a small threshold ($slope\_threshold$), indicating minimal change.
    - Curvatures are below a threshold ($curvature\_threshold$), suggesting steady trends.
- **Oscillation Detection:** Oscillations are identified by evaluating the range of recent values ($oscillation\_tolerance$). If the range is minimal, the series is deemed to be oscillating around a stable point.
- **Increasing Trend Detection**: The algorithm checks if recent slopes exceed a specified threshold ($increasing\_trend\_threshold$) to detect scenarios where the loss values are increasing rather than stabilizing.
- **Flat Change Detection:** A flat change condition is satisfied if the absolute difference between the first and last values in a recent window is smaller than a threshold ($flat\_change\_threshold$).
- **Patience Parameter:** The algorithm tracks consecutive stabilization conditions using a patience parameter ($patience$) to ensure robustness. If conditions are met for the specified number of iterations, the stabilization point is declared.

Using a statistical method provides an objective, reproducible way to determine the saturation point without requiring subjective labeling of data. The flexibility of the algorithm, with tunable thresholds for slope, curvature, and oscillation, allows it to adapt to various types of loss value series.

5. Results & Analysis

5.1 Predictive Model

For the intended application, the model with the best hallucination-based test loss metric was selected as the most appropriate choice. This decision reflects the practical requirement of allowing the model to predict sequentially, where its outputs are used as inputs for subsequent predictions. A low hallucination-based test loss signifies the model's ability to accurately extrapolate over multiple steps, a critical factor for scenarios demanding robust trend prediction.

The configuration with a learning rate of 0.001, dropout of 0.2, and embedding dimensions of 32 emerged as the best candidate with a hallucination-based test loss of 0.106. In the following the effect of some of the parameters are sketched out:

- **Learning Rate**: A smaller learning rate (e.g., 0.001) facilitated smoother convergence and improved stability, especially for long-term predictions. Larger rates (e.g., 0.01) often led to instability and suboptimal generalization.
- **Dropout**: Moderate dropout values (e.g., 0.2) effectively reduced overfitting without significantly impairing the model's learning capacity, making them particularly suited for small datasets. Higher dropout rates (e.g., 0.35) hindered the model's ability to generalize.
- **Embedding Dimensions**: Embedding dimensions significantly influenced the trade-off between capacity and overfitting. While higher dimensions (e.g., 32) offered better pattern representation, they sometimes overfit on smaller datasets. Moderate dimensions (e.g., 8) provided a better balance for this context.
- **Parameter Count**: Models with moderate parameter counts (~35,000) still sometimes outperformed larger models (~137,000) in hallucination-based testing, as their simpler structures reduced the risk of overfitting while still capturing sufficient complexity.

Learning Rate	Dropout	Embedding Dimensions	Number of Parameters	Train Loss	Test Loss (Ordinary)	Test Loss (Hallucination)
0.01	0.2	32	137601	0.016	0.006	0.204
0.01	0.2	8	35169	0.017	0.007	0.153
0.01	0.2	2	10281	0.38	0.32	0.32
0.01	0.35	32	137601	0.022	0.008	0.175
0.01	0.35	8	35169	0.037	0.012	0.147
0.01	0.35	2	10281	0.31	0.321	0.321
0.001	0.2	32	137601	0.011	0.003	0.106
0.001	0.2	8	35169	0.017	0.007	0.132

0.001	0.2	2	10281	0.26	0.319	0.32
0.001	0.35	32	137601	0.015	0.005	0.114
0.001	0.35	8	35169	0.03	0.01	0.131
0.001	0.35	2	10281	0.38	0.32	0.32

Figure 2: Results of hyperparameter tuning

Ordinary Test

The provided figures demonstrate the non-hallucinating test results, where the model predicts only a single new value at each step without using its own output as subsequent input. This controlled approach allows for an evaluation of the model's direct prediction accuracy without the cumulative effects of sequential errors, as would occur in a hallucination-based scenario. In both figures, the predicted sequences (blue) closely align with the real sequences (orange), reflecting the model's ability to effectively capture the underlying patterns in the data. The bottom section visualizes the confidence scores, derived from dropout variability during inference. The confidence remains relatively stable across the sequence, often exceeding the pre-defined threshold of 0.65 (indicated by the dashed red line).

In the first figure, the alignment between predicted and real sequences is very close throughout the range, with even lower deviation and high confidence scores. In the second figure, there is a greater deviation in some parts of the sequence, though the predictions stabilize and align with the real sequence as the index progresses. The confidence scores reflect this, remaining above the threshold despite some fluctuations.

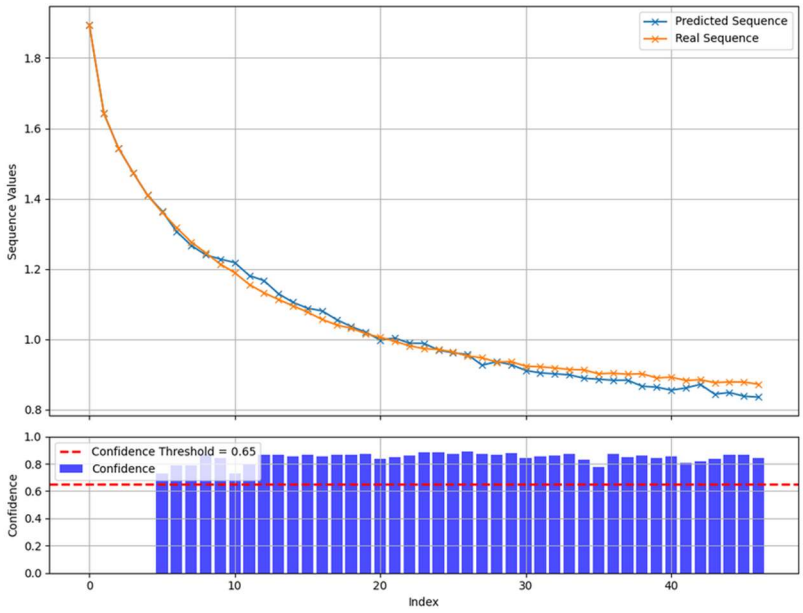

Figure 3: Ordinary test I

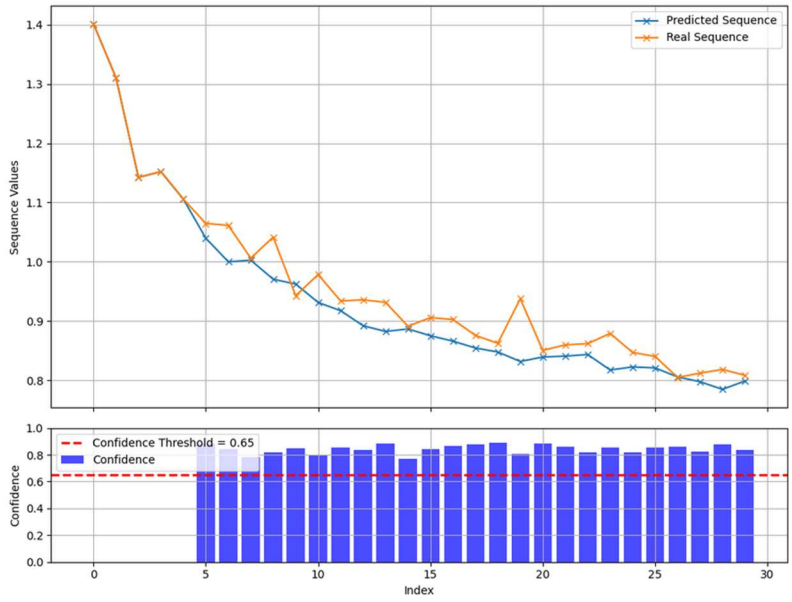

Figure 4: Ordinary test II

Hallucinating Test

The provided figures illustrate the model's hallucination-based predictions, where, after the red vertical line, it uses its own outputs as inputs for subsequent predictions. The model starts with only five known values, meaning the fewer initial values it has, the more it must rely on self-generated inputs, increasing the risk of compounding errors and instability. Despite some deviations from the real sequence (orange), the predicted sequence (blue) follows the overall trend effectively, demonstrating the model's ability to generalize patterns even in extended predictions.

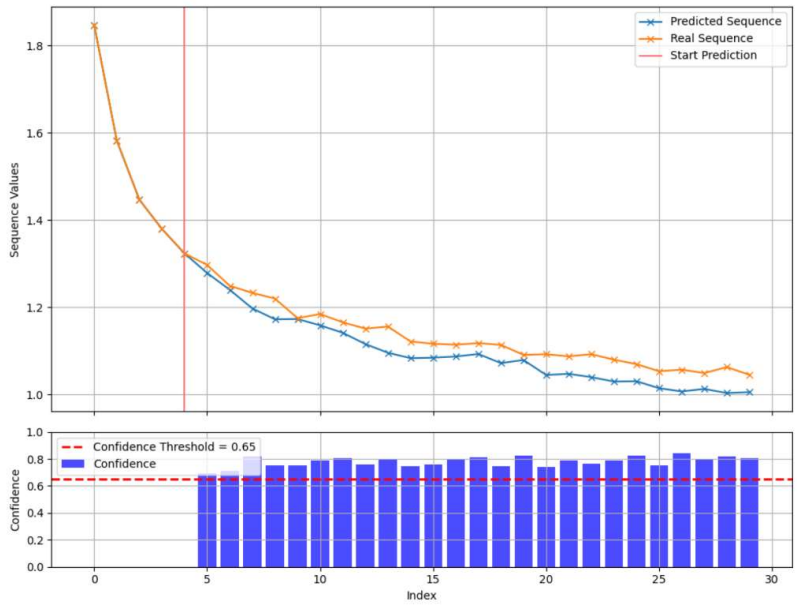

Figure 5: Hallucinating test I

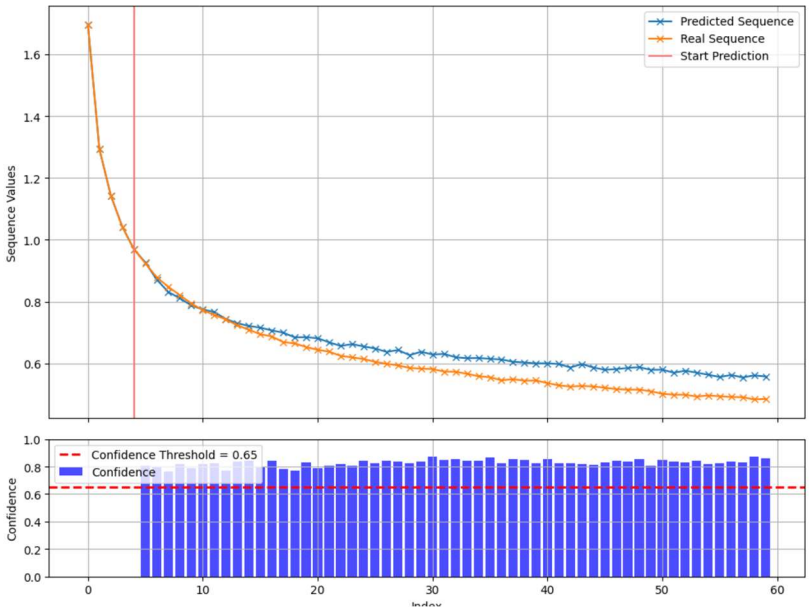

Figure 6: Hallucinating test II

The figures below show hallucination-based predictions where the model struggles to maintain accuracy. After the red line, deviations from the real sequence grow progressively as the model uses its own outputs as inputs. While the predicted sequences follow the overall trend, the increasing divergence highlights difficulties in long-term extrapolation.

In the figure below, the model's uncertainty is reflected in a lower confidence score, especially at the beginning of the prediction where it wrongly predicts a downward trend. In the other figure on the other hand, the confidence scores remain high and surpassing the threshold of 0.65. These two examples show that the model also has limitations in extended predictions, suggesting the need for improvements to mitigate cumulative errors in extended sequences.

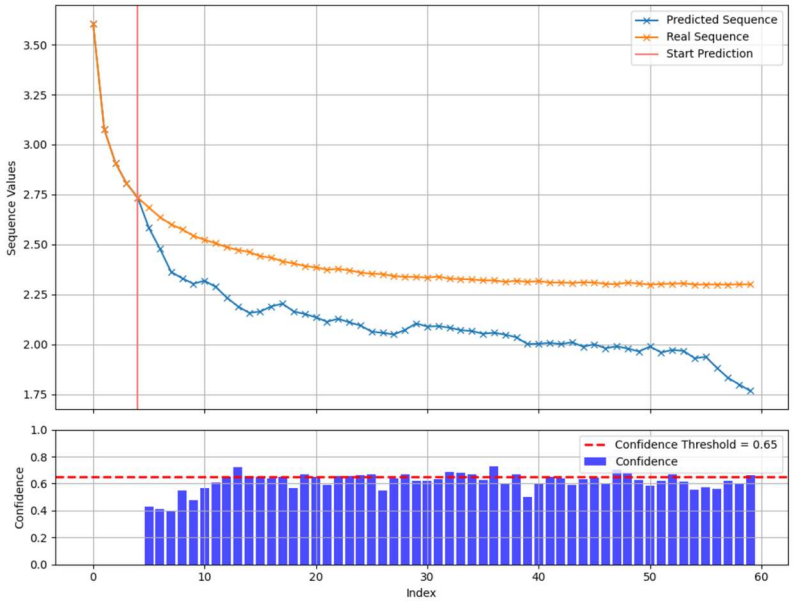

Figure 7: Hallucinating test IV

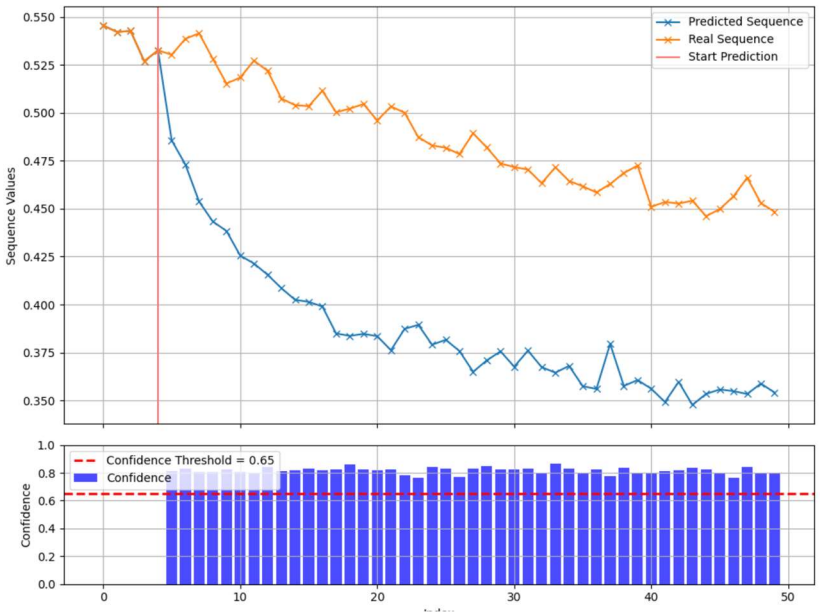

Figure 8: Hallucinating test III

5.2 Stabilization Point Detection

5.2.1 Parameters

After experimentation and manual tuning, the following parameters were found to detect stabilization points effectively. However, it is important to note that this process remains inherently subjective, as the definition of stabilization can vary depending on the specific training dynamics and desired outcomes. The tuning process involved systematically adjusting one parameter at a time while holding the others constant, allowing for the isolation of each parameter's effect on the algorithm's behavior. This iterative approach was necessary given the high dimensionality of the parameter space and the interdependence between factors such as slope, curvature, and oscillation tolerance.

- **Slope Threshold**: 0.005. This threshold defines the maximum rate of change in the loss values for stabilization. A low value ensures that only near-zero slopes are considered, capturing flat regions in the loss curve.
- **Curvature Threshold**: 0.05 The curvature threshold assesses the acceleration or deceleration in the loss values. By limiting this value, the algorithm identifies regions where the slope changes minimally, indicating stabilization.
- **Patience**: 1 iteration. This parameter allows the algorithm to confirm stabilization conditions for at least one consecutive window. A value of 1 ensures quick detection of stabilization without requiring prolonged flatness.
- **Oscillation Tolerance**: 0.001. This parameter accounts for minor fluctuations in loss values. A low tolerance ensures that only minimal oscillations around a stable point are considered as indicative of saturation.
- **Increasing Trend Threshold**: 0.01. This threshold detects increasing trends in loss values. If the slope exceeds this value, the region is classified as non-stabilized.

- **Flat Change Threshold**: 0.003. This threshold compares the absolute change in loss values over a window. If the change is below this value, it suggests stabilization.

These parameters were chosen to balance sensitivity to minor trends while ensuring robustness in detecting true stabilization points. The aim was to identify when the loss progression slows sufficiently to indicate that further training would provide diminishing returns.

5.2.2 Examples

Figures 1 and 2 illustrate cases where identifying a saturation point or halting training would not be appropriate, as the loss curves show a consistent downward trend, indicating ongoing improvements. In these scenarios, the key parameters influencing the algorithm's decision include the slope threshold (0.005), which remained unmet due to the continuous decrease in loss, and the flat change threshold (0.003), which was exceeded across all windows. These results highlight that the loss progression still reflects meaningful learning, and it would not make sense to classify these cases as stabilized or saturated.

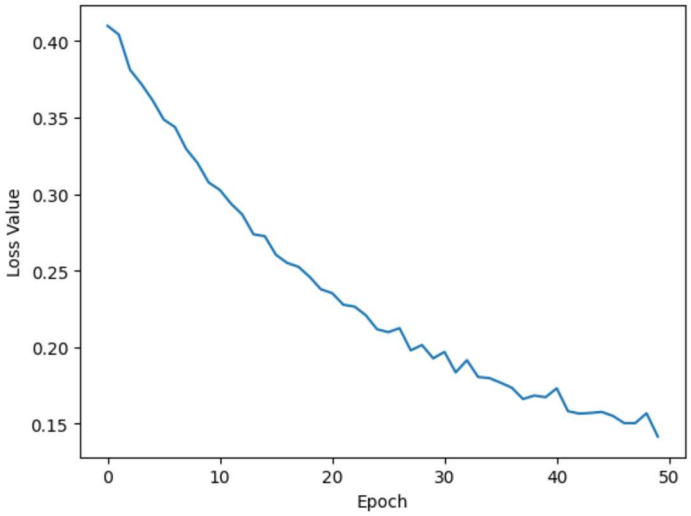

Figure 7: Example stabilization point I

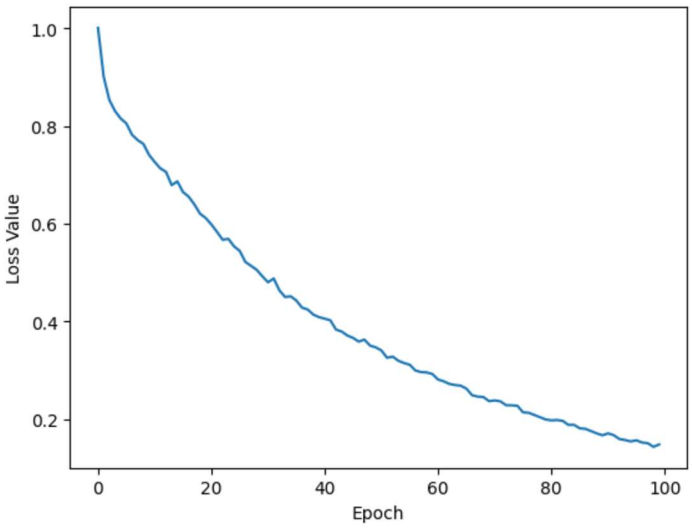

Figure 8: Example stabilization point II

Figures 11 & 12 illustrate cases where the algorithm successfully identified appropriate saturation points in the loss curves. In both instances, the loss progression exhibits a clear reduction followed by a plateau, indicating that further training would yield diminishing returns.

The identified saturation points are marked with a red dashed line and correspond to epochs where the stabilization criteria were met. Specifically, the slope (0.005) and curvature (0.05) thresholds fell below the defined limits, indicating minimal change in the loss values over successive epochs. Additionally, the flat change threshold (0.003) confirmed that the variation in loss across the stabilization window was negligible, and no increasing trends (0.01) were detected.

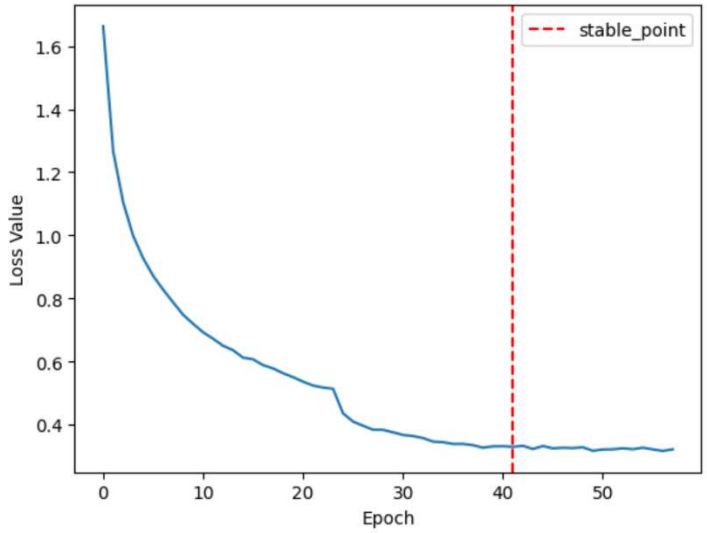

Figure 9: Example stabilization point III

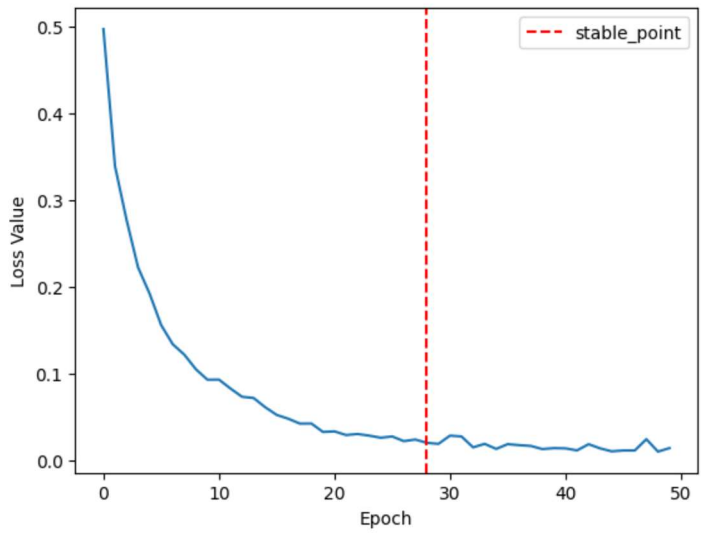

Figure 10: Example stabilization point IV

Figure 13 demonstrates a case where the algorithm successfully terminated training due to an increasing trend in the loss values. After an initial sharp decline, the loss reaches a minimum around epoch 20, after which it begins to increase steadily. The algorithm correctly identified this upward trend, meeting the increasing trend threshold (0.01) and halting further training.

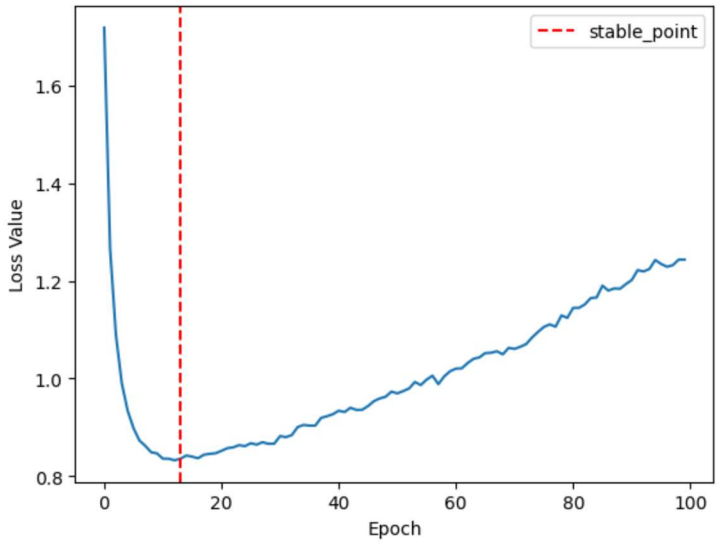

Figure 11: Example stabilization point V

Figure 14 illustrates a case where the algorithm prematurely identified a saturation point. While the loss curve initially flattens, suggesting stabilization, a significant drop in the loss value occurs around 40 epochs later. This indicates that the stabilization criteria were met too early, leading to an incorrect conclusion about the training's progress.

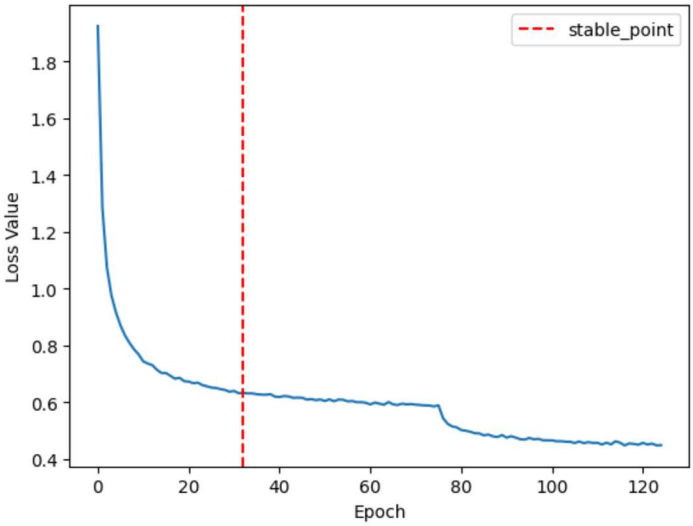

Figure 12: Example stabilization point VI

5.3 Integrating Stabilization Detection with Predictive Modeling

The provided plots illustrate the combined application of the stabilization detection algorithm on the hallucinated sequences generated by the predictive model. By leveraging the hallucinated sequence, the system can preemptively determine a saturation point, allowing for an earlier decision on when training is no longer useful. This approach reduces unnecessary training epochs, and thus eliminates wasteful computation.

As previously observed in the non-hallucinating test results, the model performs more accurately when provided with longer input sequences, leading to better prediction of loss trends. This means that as the accuracy of hallucinated sequences improves, the ability to detect true saturation points becomes increasingly precise.

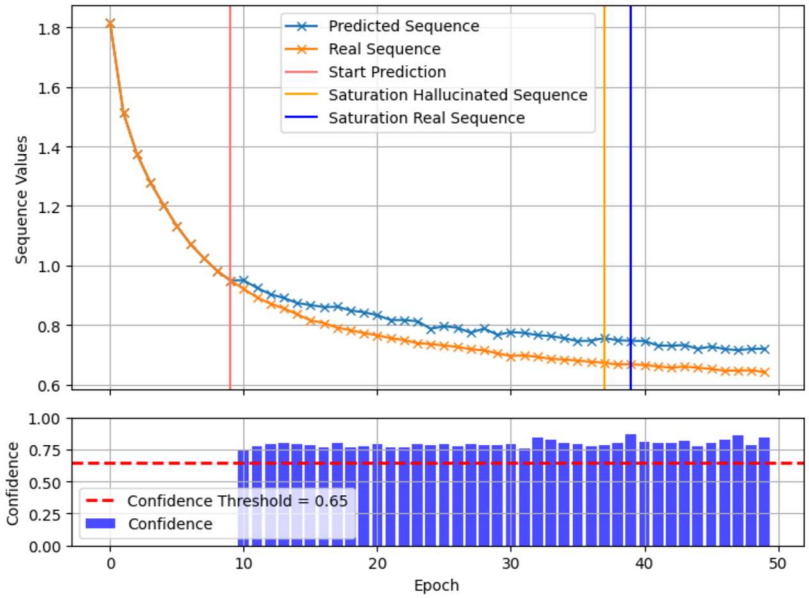

Figure 13: Forecasting & stabilization detection combined I

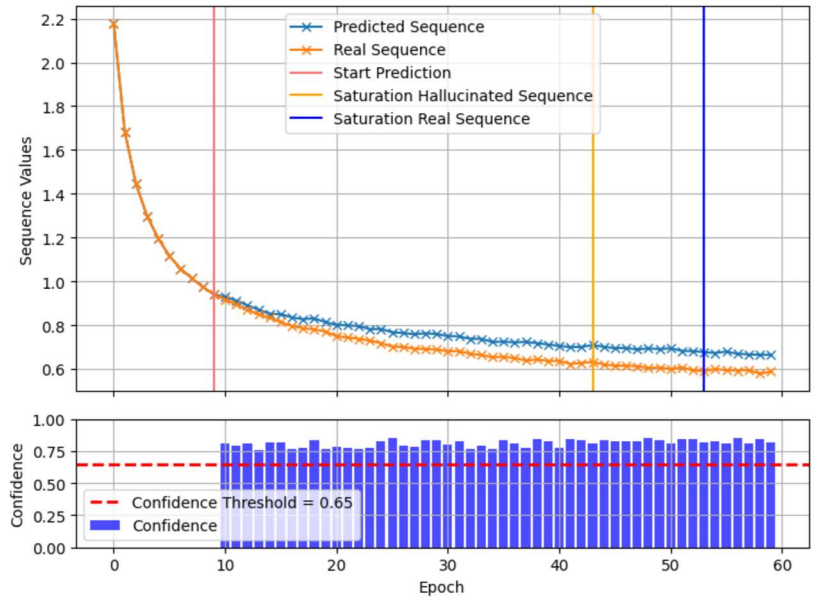

Figure 14: Forecasting & stabilization detection combined II

5.4 Enhanced Framework

The new modular framework enhances flexibility and robustness, addressing limitations of the original D-PoDL system, where the structure lacked modularity and was tightly coupled to specific tasks. The updated design allows for seamless replacement of tasks (e.g., training on CIFAR-10 or MNIST dataset) and the prediction model used for future loss value calculations. This modularity simplifies adaptation to different use cases and promotes scalability. A class

and method diagram of the enhanced system is provided in the Appendix for further clarification of the structural and functional components.

The workflow is now structured as follows:

1. **Task Initialization**: Each deep learning task (e.g., CIFAR-10, MNIST) is implemented via a unified TaskInterface, which standardizes model creation, data loading, training, and metric tracking. This modular interface allows easy integration of new tasks or datasets without requiring structural changes to the framework.
2. **Proof-of-Work Validation:** Before initiating the D-PoDL process, a lightweight Proof-of-Work (PoW) mechanism (pre_pow) verifies the hash integrity from the previous block. This step is designed to ensure security and computational integrity consistent with blockchain requirements.
3. **Distributed Proof-of-Deep-Learning Solver:** The D-PoDL solver (dpodl_solver) serves as the central processing unit. Its main responsibilities include:
    - Training the assigned deep learning model while tracking performance metrics such as accuracy and loss.
    - Monitoring stabilization points using a dynamic predictor model to identify when further training is no longer "useful work."
    - Maintaining computational difficulty through integration with the hash-to-hash architecture, ensuring the system remains secure and distributed.
4. **Stabilization Point Detection:** After each iteration, the stabilization detection algorithm evaluates the progression of the loss values. Ou predictor model analyzes the sequence of loss valuess and forecasts future trends. The system then determines whether further training still qualifies as "useful work."
5. **Confidence Assessment:** If the stabilization point is not reached, the system assesses the model's confidence in continuing to "hallucinate" or extrapolate further predictions. Confidence is calculated using dropout-based variability and normalized through a dynamic threshold.
6. **Dynamic Switching or Halting:** If a stabilization point is confirmed the task concludes, otherwise it continues training for another iteration and repeats the process. The framework then hashes the output, ensuring cryptographic integrity, and dynamically switches to another dataset or architecture if applicable.
7. **Post-Training Validation:** Once the training process concludes, the framework validates the model's performance and computational integrity through a post-check mechanism. This ensures that the results meet predefined accuracy and stability thresholds, confirming that the computational effort qualifies as "useful work."

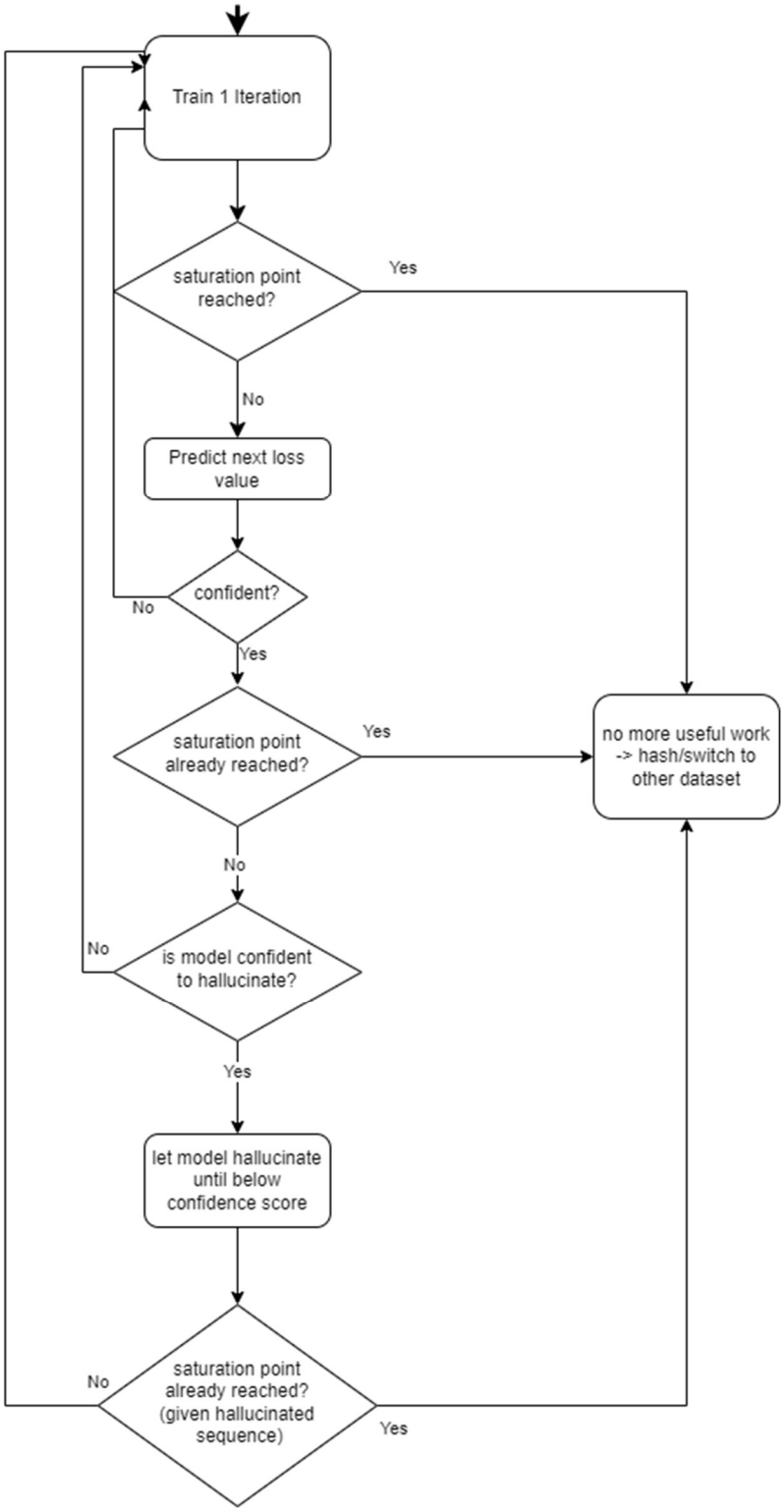

Figure 15: Flowchart of system

6. Discussion & Conclusion

This project successfully refined the concept of useful work in deep learning training within PoUW systems. By integrating stabilization point detection and a Transformer-based predictive model, it introduced a dynamic approach to determining when training stops to be useful. This

method eliminates the inefficiencies of fixed accuracy thresholds by detecting non-linear training behaviors such as plateaus and diminishing returns, improving the use of computational resources. The experimental results validated this approach, and demonstrated solid performance in both ordinary and hallucination-based testing. The Transformer model effectively identified loss progression trends, allowing proactive termination of training when further iterations would no longer improve its capabilities. This represents a major improvement over static threshold-based methods, making the D-PoDL framework more adaptable, scalable, and efficient. By integrating dynamic loss-based evaluation, this study directly addresses the research question: "*How can we determine when deep learning training remains useful work in D-PoDL systems?*" The findings contribute to a broader understanding of how computational work can be optimized in deep learning tasks.

While the results were promising, some limitations remain. The dataset used for training the predictive model lacked sufficient examples of increasing loss functions, which may impact its generalizability. This bias reflects the reality that such training runs are often terminated early, leading to a scarcity of data for these scenarios. One potential solution is the generation of synthetic loss curves to improve robustness. Another challenge lies in the task-specific nature of the predictive model. Currently a separate predictor is required for each task, reducing its scalability. A more unifying approach could involve integrating explicit task-related parameters, such as dataset characteristics or model architecture, into the predictive model, allowing it to adapt dynamically to various tasks.

An additional challenge is double descent in training dynamics, where loss initially stagnates or worsens before improving again [8]. The current stabilization detection may prematurely terminate training during this phase, potentially missing later improvements. Future research could explore methods to distinguish between true saturation and temporary stagnation.

Looking ahead, the D-PoDL framework could be enhanced by introducing a multi-task optimization mechanism. A resource-aware scheduling system could be implemented, tracking the FLOPS spent on useful work. Once a threshold is reached for a given task, computational resources could be reallocated to another, creating a more dynamic and efficient system that better distributes work across multiple deep learning trainings in a decentralized setting.

By introducing a more flexible and adaptive framework for detecting useful work in deep learning training, this study improves the efficiency of D-PoDL, reducing computational waste without compromising security or decentralization. These advancements contribute to making blockchain mining more purpose-driven, and resource-efficient.

7. Bibliography

[1] M. L. K. T. Xiangyu Su, "Provably Secure Blockchain Protocols from Distributed Proof-of-Deep-Learning," *Cryptology ePrint Archive,* p. Paper 2023/1059, 2023.

[2] S. Nakomoto, "Bitcoin: A peer-to-peer electronic cash system," 2008.

[3] A. d. Vries, "Bitcoin's growing energy problem," *Joule,,* no. 2(5), pp. 801-805, 2018.

[4] V. Buterin, "A next-generation smart contract and decentralized application platform," Ethereum White Paper, 2018.

[5] A. C. T. S. S. P. A. D. Ushnish Sarkar, "Enhancing ASL Recognition with GCNs and Successive Residual Connections," *To be submitted in G2-SP CV,* 2024.

[6] A. C. C. Y. B. Ian Goodfellow, "Regularization," in *Deep Learning*, 2016, pp. 224-270.

[7] N. S. N. P. J. U. L. J. A. N. G. L. K. I. P. Ashish Vaswani, "Attention Is All You Need," *Proc. NeurIPS,* no. 30, p. 5998–6008, 2017.

[8] J. S. Sepp Hochreiter, "Long Short-Term Memor," *Neural Computation,* no. 9(8), p. 1735–1780, 1997.

[9] D. H. S. M. S. M. Mikhail Belkina, "Reconciling modern machine-learning practice and the classical bias-variance trade-off," *Proceedings of the National Academy of Sciences,* no. 116(32), p. 15849–15854, 2019.

8. Appendix

The full implementation, including the predictive model, stabilization detection algorithm, and D-PoDL integration, is available on GitHub.

GitHub Repository: https://github.com/vincent-stadler/dPoDL

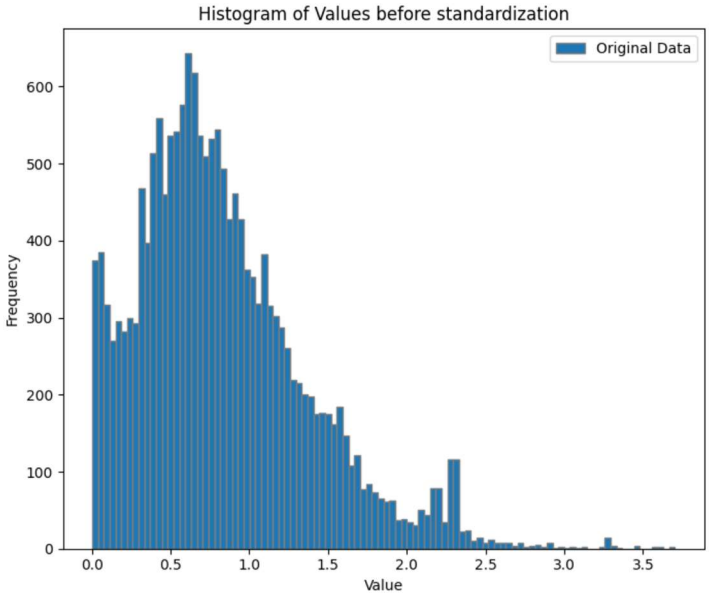

Figure 16: Histogram of loss values of used dataset

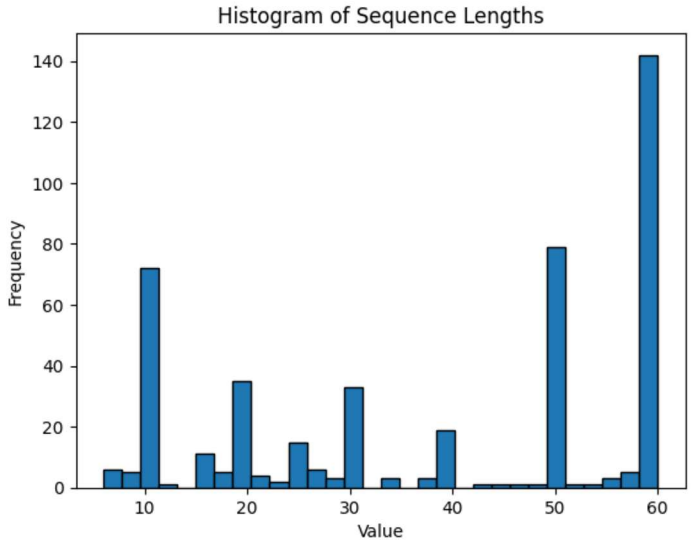

Figure 17: Histogram of sequence length of used dataset

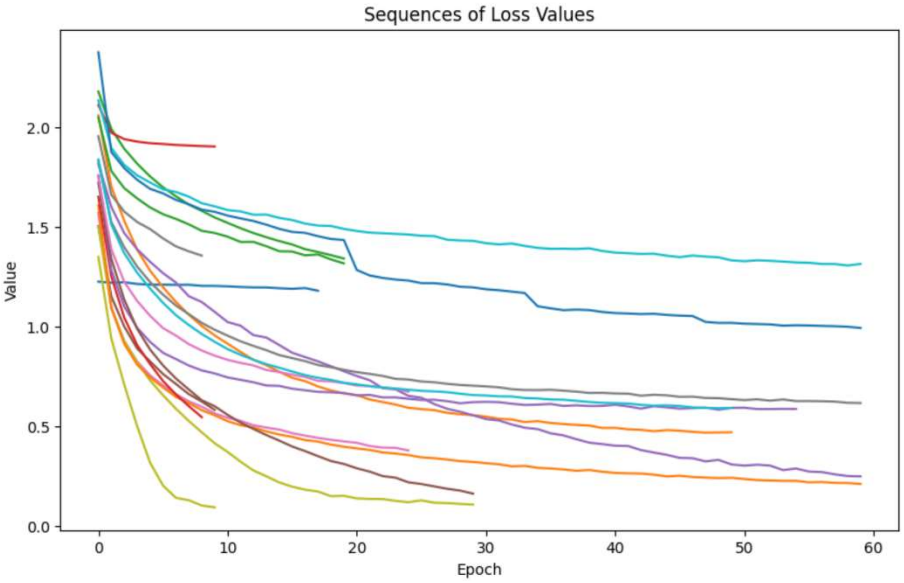

Figure 18: Sample sequences of used dataset

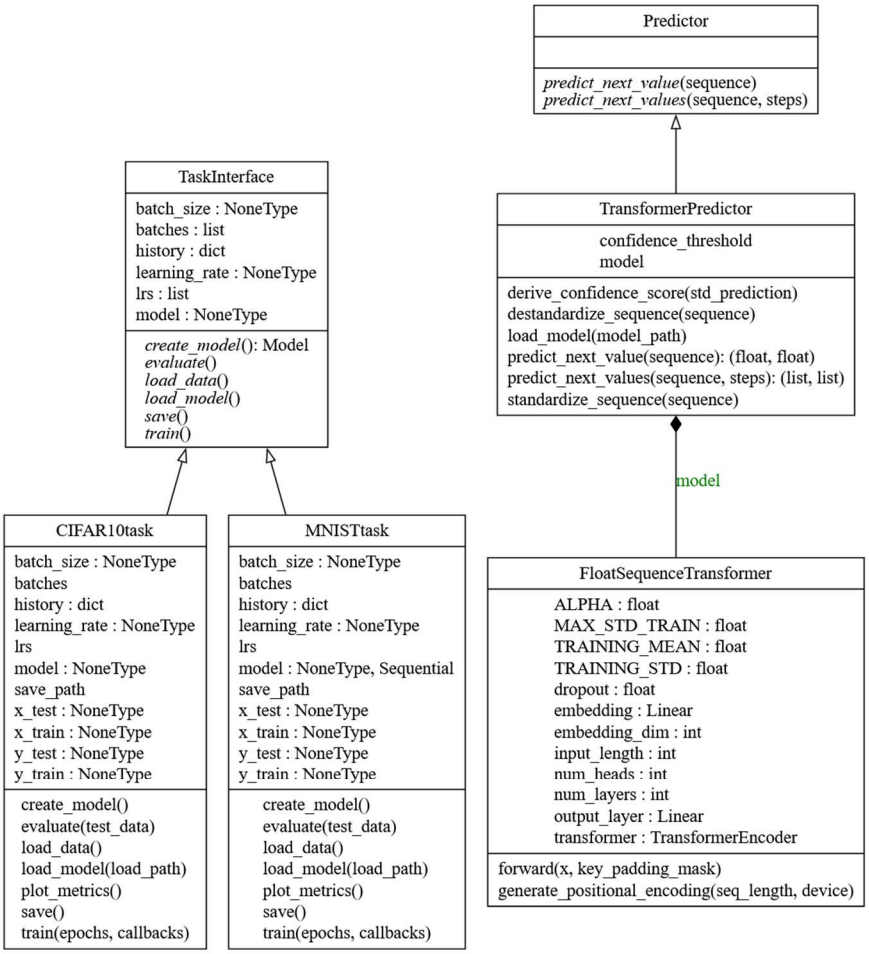

Figure 19: Class diagram of enhanced code structure

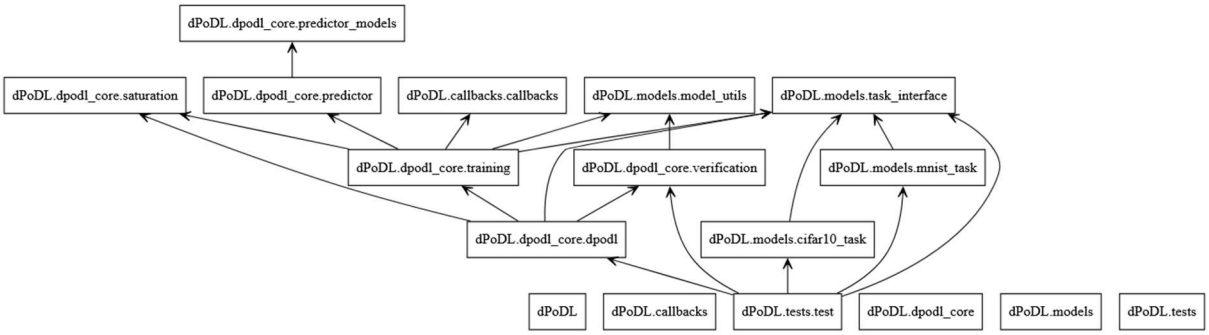

Figure 20: Interaction between methods of enhanced code structure